

1. Processid

```
#include<stdio.h>
#include<unistd.h>
int main()
{
printf("Process ID =%d\n",getpid());
return 0;
}
```

Output :

```
hp@Hima:~/OS_System$ nano display.c
hp@Hima:~/OS_System$ gcc display.c -o display
hp@Hima:~/OS_System$ ./display
Process ID =16172
```

2. creating child using fork()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

Output:

```
hp@Hima:~$ nano parentid.c
hp@Hima:~$ gcc parentid.c -o parent
hp@Hima:~$ ./parent
Parent PID = 990
Child PID = 991
Parent PID = 991
```

Parent Waits for Child (wait())

```
GNU nano 8.7.1 wait.c
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
    pid_t pid = fork();
    if(pid==0){

        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else{
        wait(NULL);
        printf("Parent Running After Child\n");
    }
    return 0;
}
```

Output:

```
hp@Hima:~$ nano wait.c
hp@Hima:~$ gcc wait.c -o wait
hp@Hima:~$ ./wait
Child Process Executing...
Child Finished
Parent Running After Child
```

Multiple Child Processes

```
GNU nano 8.7.1 wait.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i=1; i<=3; i++)
    {
        if(fork() == 0)
        {
            printf("Child %d PID = %d\n", i, getpid());
            return 0;
        }
    }

    return 0;
}
```

Output:

```
hp@Hima:~$ nano wait.c
hp@Hima:~$ nano wait.c
hp@Hima:~$ gcc wait.c -o wait
hp@Hima:~$ ./wait
Child 1 PID = 868
Child 2 PID = 869
Child 3 PID = 870
hp@Hima:~$ |
```

Execute linux command using execlp()

```
GNU nano 8.7.1 wait.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls", "ls", "-l", NULL);

    printf("This will not execute.\n");

    return 0;
}
```

Output:

```
hp@Hima:~$ nano wait.c
hp@Hima:~$ gcc wait.c -o wait
hp@Hima:~$ ./wait
Before exec()
total 108
-rw-r--r-- 1 hp hp 28 Aug 5 08:15 Hello
drwxr-xr-x 2 hp hp 4096 Jul 29 06:05 OS-System
-rw-r--r-- 1 hp hp 0 Jul 29 05:44 OS-System.txt
drwxr-xr-x 2 hp hp 4096 Jul 30 04:18 OS_System
-rw-r--r-- 1 hp hp 13 Aug 6 04:14 file1.txt
-rwxr-xr-x 1 hp hp 15992 Aug 1 05:44 hello
-rw-r--r-- 1 hp hp 167 Aug 1 05:52 hello.c
-rw----- 1 hp hp 9 Aug 1 05:38 nano.554.save
-rw-r--r-- 1 hp hp 13 Aug 12 06:49 output.txt
-rwxr-xr-x 1 hp hp 16040 Aug 1 05:53 parent
-rw-r--r-- 1 hp hp 211 Aug 1 05:54 parentid.c
-rwxr-xr-x 1 hp hp 16176 Aug 6 04:18 partical1
-rw-r--r-- 1 hp hp 254 Aug 12 07:05 partical1.c
-rwxr-xr-x 1 hp hp 15992 Aug 12 07:57 wait
-rw-r--r-- 1 hp hp 179 Aug 12 07:56 wait.c
-rw-r--r-- 1 hp hp 190 Aug 1 06:07 wait.c.save
```

Parent and Child Executing Concurrently

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();

    for(int i=1; i<=5; i++)
    {
        printf("PID: %d Count: %d\n", getpid(), i);
        sleep(1);
    }

    return 0;
}
```

Output :

```
hp@Hima:~$ nano wait.c
hp@Hima:~$ gcc wait.c -o wait
hp@Hima:~$ ./wait
PID: 973 Count: 1
PID: 974 Count: 1
PID: 973 Count: 2
PID: 974 Count: 2
PID: 973 Count: 3
PID: 974 Count: 3
PID: 974 Count: 4
PID: 973 Count: 4
PID: 973 Count: 5
PID: 974 Count: 5
hp@Hima:~$
```

Zombie Process

```
hp@Hima: ~
GNU nano 8.7.1 wait.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Exiting\n");
        return 0;
    }
    else
    {
        sleep(15);
        printf("Parent Finished\n");
    }

    return 0;
}
```

Output

```
hp@Hima:~$ nano wait.c
hp@Hima:~$ gcc wait.c -o wait
hp@Hima:~$ ./wait
Child Exiting

Parent Finished
hp@Hima:~$
```

OrphanProcess :

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

Output

```
hp@Hima: ~
hp@Hima:~$ nano wait.c
hp@Hima:~$ gcc wait.c -o wait
hp@Hima:~$ ./wait
Parent Exiting
hp@Hima:~$ Child PID : 1152
New Parent PID : 1086
```

